# Compte rendu — TP Android : gestion des contacts (Version 3)

**Nom et prénom :** ..... **Groupe :** .....

## 1. Architecture des fragments

L'application ne contient plus qu'une activité, `MainActivity`, qui héberge un `FragmentContainerView`. Trois fragments dynamiques remplacent les écrans de la V2 : `ListeContactsFragment`, `DetailContactFragment` et `CarteContactFragment`. La navigation utilise `supportFragmentManager.commit { replace(...); addToBackStack(...) }` : le bouton Retour (ou la flèche de la barre) dépile la transaction et revient à l'écran précédent. Au premier lancement seulement (`savedInstanceState == null`), la liste est ajoutée ; après une rotation le `FragmentManager` restaure lui-même la pile, ce qui évite les fragments en double.

## 2. Communication entre fragments

- **Aller vers le détail** : `DetailContactFragment.nouvelleInstance(id)` place l'identifiant dans un `Bundle` (`arguments`).
- **Actualiser la liste** : choix d'un **ViewModel partagé** (`by activityViewModels()`). Il conserve les contacts dans une `MutableLiveData`, survit à la rotation, et `mettreAJour(contact)` publie une nouvelle liste observée par la liste. Ce choix a été préféré à la *Fragment Result API* car la même donnée sert à trois écrans et doit rester disponible après une recréation.
- **Aller vers la carte** : nom, latitude et longitude saisis sont passés dans un `Bundle`.

## 3. RecyclerView

`ContactAdapter` hérite de `ListAdapter<Contact, ContactViewHolder>` : le `ViewHolder` garde les références des vues d'une ligne, `DiffUtil` compare les contacts par `id` puis par contenu et ne redessine que les lignes changées. Le clic transmet le `Contact` lui-même (donc son `id`).

## 4. Stratégie de filtrage

`ContactViewModel.filtrer(liste, texte)` renvoie les contacts dont le nom, le prénom, « prénom nom » ou le téléphone (sans espaces) contiennent le texte, avec `ignoreCase = true`. Le texte du filtre est mémorisé dans le ViewModel. Comme l'adapter reçoit la liste filtrée et renvoie l'objet cliqué, la position dans la liste filtrée n'est jamais utilisée pour retrouver un contact : **pas d'erreur d'indice**.

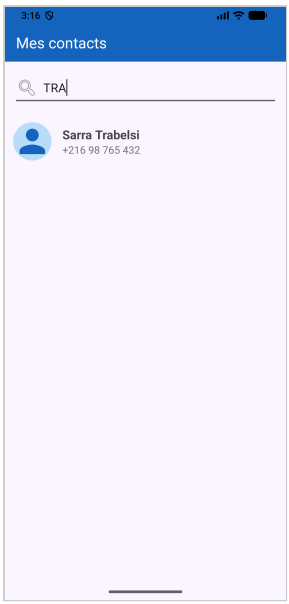
## 5. Coordonnées GPS et Google Maps

`Contact` a deux propriétés `latitude: Double?` et `longitude: Double?`. Dans le détail, deux champs permettent de les saisir ; la validation accepte « aucune position » (deux champs vides) ou deux nombres avec latitude ∈ [−90 ; 90] et longitude ∈ [−180 ; 180], sinon un message d'erreur est affiché sous chaque champ et rien n'est enregistré.

`CarteContactFragment` contient un `SupportMapFragment` enfant ; `getMapAsync` ajoute un marqueur titré avec le nom du contact et centre la caméra (`newLatLngZoom`, zoom 13). Si les coordonnées sont absentes, un message explicite remplace la carte. La **clé API** est lue dans `local.properties` (non versionné) et injectée par `manifestPlaceholders["MAPS_API_KEY"]` : elle n'apparaît jamais dans GitHub. Si elle manque, l'écran l'indique.

## 6. Tests

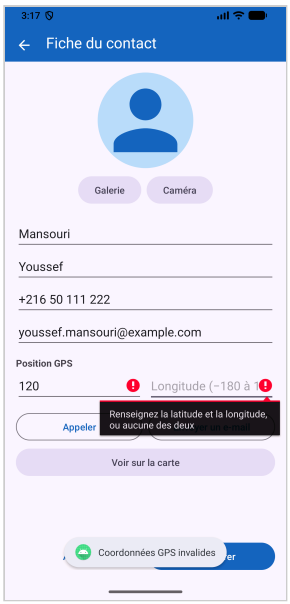
| Test | Résultat |
|---|---|
| Filtre « TRA » (majuscules) → Sarra Trabelsi | ✅ |
| Filtre sur le téléphone « 55123 » → Mohamed Gharbi | ✅ |
| Filtre sans résultat → message | ✅ |
| Contact sélectionné dans une liste filtrée → bonne fiche | ✅ |
| Coordonnées hors limites / champ manquant → erreurs, carte non ouverte | ✅ |
| Contact sans position → message sur l'écran carte | ✅ |
| Modification téléphone + coordonnées → liste actualisée | ✅ |
| Carte avec marqueur | ⚠️ nécessite une clé Google Maps personnelle (message « clé absente » vérifié) |

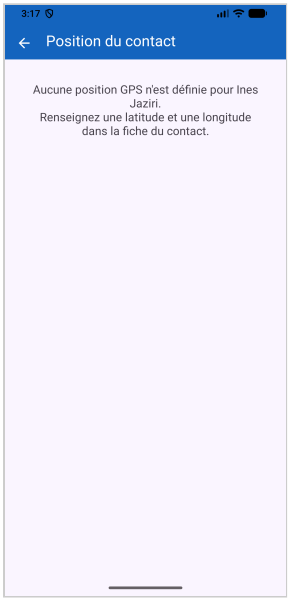









## 7. Questions de compréhension

- **Activity / Fragment** : une activité est un écran géré par le système ; un fragment est une portion d'interface réutilisable, avec son propre cycle de vie, qui vit à l'intérieur d'une activité.
- **FragmentContainerView et addToBackStack()** : le premier est le conteneur recommandé (gère correctement les animations et l'ordre d'affichage) ; `addToBackStack` enregistre la transaction pour que Retour l'annule.
- **Adapter / ViewHolder** : l'adapter crée et remplit les lignes à partir des données ; le ViewHolder conserve les vues d'une ligne recyclée pour éviter des `findViewById` répétés.
- **Erreur d'indice après filtrage** : ne jamais utiliser la position pour indexer la liste complète ; passer l'objet ou son identifiant.
- **Identifiant plutôt que position** : la position change avec le filtre, le tri ou une suppression ; l'identifiant est stable.
- **Protéger la clé Maps** : la placer dans `local.properties` non versionné, l'injecter au build et restreindre la clé (package + empreinte SHA-1) dans Google Cloud.

## 8. Difficultés

- Ne pas écraser une saisie en cours en revenant de la carte : le fragment détail mémorise qu'il a déjà rempli ses champs.
- La flèche de retour restait affichée après un retour système (geste prédictif) : sa mise à jour est maintenant différée après la fin de la transaction.
- Pas de clé Google Maps disponible pour la démonstration : un message explicite a été prévu.